

AI Video Pluggable Component – Complete Updated Files

src/app/ai-video-agent/emotion.model.ts

```
export type EmotionType = 'EXPLAIN' | 'APPROVE' | 'ATTENTION';

export const EMOTION_VIDEO_MAP: Record<EmotionType, string> = {
  EXPLAIN: '/videos/explaining.mp4',
  APPROVE: '/videos/thumbs_up.mp4',
  ATTENTION: '/videos/hand_raise.mp4'
};
```

src/app/ai-video-agent/emotion.service.ts

```
import { Injectable } from '@angular/core';
import { EmotionType } from './emotion.model';

@Injectable({ providedIn: 'root' })
export class EmotionService {

  // 🔥 Azure OpenAI Details
  private endpoint = 'YOUR_AZURE_OPENAI_ENDPOINT';
  private deployment = 'YOUR_DEPLOYMENT_NAME';
  private apiVersion = 'YOUR_API_VERSION';
  private apiKey = 'YOUR_AZURE_OPENAI_KEY';

  async analyze(text: string): Promise<EmotionType[]> {

    const sentences =
      text.match(/[^\.\!\?]+\.[\!\?]+|[^\.\!\?]+$/g)?.map(s => s.trim()) || [text];

    try {

      const url =
        `${this.endpoint}/openai/deployments/${this.deployment}/chat/completions?api-version=${this.apiVersion}`;

      const res = await fetch(url, {
        method: 'POST',
```

```

    headers: {
      'Content-Type': 'application/json',
      'api-key': this.apiKey
    },
    body: JSON.stringify({
      temperature: 0,
      messages: [
        {
          role: 'system',
          content: `
You are an emotion classifier.

Return ONLY JSON array.

Allowed labels:
- EXPLAIN
- APPROVE
- ATTENTION

Rules:
- Output ONLY valid JSON
- No markdown
- No explanation
- Array length MUST exactly match number of input sentences
`,
        },
        {
          role: 'user',
          content: sentences.join('\n')
        }
      ]
    })
  });

  const data = await res.json();

  const raw =
    data?.choices?.[0]?.message?.content?.trim() ?? '';

  const match = raw.match(/\[.*\]/s);

  if (!match) {
    return this.fallback(sentences);
  }

  const parsed = JSON.parse(match[0]).map((e: any) => {

    const val = String(e).trim().toUpperCase();

```

```

        if (val === 'APPROVE') return 'APPROVE';

        if (val === 'ATTENTION') return 'ATTENTION';

        return 'EXPLAIN';
    });

    // 🔥 length safety
    return sentences.map((_, i) => parsed[i] ?? 'EXPLAIN');

} catch (err) {

    console.error('Emotion API failed:', err);

    return this.fallback(sentences);
}

private fallback(sentences: string[]): EmotionType[] {

    return sentences.map(s => {

        if (/great|awesome|excellent|good|well done/i.test(s)) {
            return 'APPROVE';
        }

        if (/important|attention|warning|critical|note/i.test(s)) {
            return 'ATTENTION';
        }

        return 'EXPLAIN';
    });
}
}

```

src/app/ai-video-agent/tts.service.ts

```

import { Injectable } from '@angular/core';
import * as SpeechSDK from 'microsoft-cognitiveservices-speech-sdk';

@Injectable({ providedIn: 'root' })
export class TtsService {

```

```

private speechConfig: SpeechSDK.SpeechConfig;

constructor() {

    this.speechConfig = SpeechSDK.SpeechConfig.fromSubscription(
        'YOUR_AZURE_SPEECH_KEY',
        'YOUR_AZURE_REGION'
    );

    this.speechConfig.speechSynthesisVoiceName =
        'en-US-AriaNeural';
}

speak(text: string): Promise<ArrayBuffer> {

    const synthesizer =
        new SpeechSDK.SpeechSynthesizer(
            this.speechConfig,
            undefined
        );

    return new Promise((resolve, reject) => {

        synthesizer.speakTextAsync(
            text,
            (result) => {
                resolve(result.audioData);
                synthesizer.close();
            },
            (err) => {
                synthesizer.close();
                reject(err);
            }
        );
    });
}
}

```

src/app/ai-video-agent/sync-engine.service.ts

```

import { Injectable, signal } from '@angular/core';
import { EmotionType, EMOTION_VIDEO_MAP } from './emotion.model';
import { TtsService } from './tts.service';

```

```

@Injectables({ providedIn: 'root' })
export class SyncEngine {

  isPaused = signal(false);
  isStopped = signal(false);

  async play(
    sentences: string[],
    emotions: EmotionType[],
    tts: TtsService,
    switchVideo: (src?: string, stop?: boolean) => void,
    preload: (src: string) => void,
    emitText: (t: string) => void
  ) {

    for (let i = 0; i < sentences.length; i++) {

      if (this.isStopped()) {
        switchVideo('', true);
        break;
      }

      while (this.isPaused()) {
        await this.sleep(50);
      }

      const emotion = emotions[i] ?? 'EXPLAIN';
      const src = EMOTION_VIDEO_MAP[emotion];

      const nextEmotion = emotions[i + 1] ?? emotion;
      preload(EMOTION_VIDEO_MAP[nextEmotion]);

      // 🗣 generate audio first
      const audioBuffer = await tts.speak(sentences[i]);

      emitText(sentences[i]);

      const ctx = new AudioContext();
      const decoded = await ctx.decodeAudioData(audioBuffer.slice(0));

      const source = ctx.createBufferSource();
      source.buffer = decoded;
      source.connect(ctx.destination);

      // 🎬 video slightly before audio
      switchVideo(src);

      await this.sleep(40);
    }
  }
}

```

```

        source.start();

        await new Promise<void>((res) => {
            source.onended = () => {
                ctx.close();
                res();
            };
        });

        switchVideo('', true);
    }

    private sleep(ms: number) {
        return new Promise(res => setTimeout(res, ms));
    }
}

```

src/app/ai-video-agent/ai-video-agent.component.ts

```

import {
    Component,
    ElementRef,
    ViewChild,
    input,
    output,
    signal,
    effect
} from '@angular/core';

import { EmotionService } from '../emotion.service';
import { TtsService } from '../tts.service';
import { SyncEngine } from '../sync-engine.service';

@Component({
    selector: 'ai-video-agent',
    standalone: true,
    templateUrl: './ai-video-agent.component.html',
    styleUrls: ['./ai-video-agent.component.scss']
})
export class AiVideoAgentComponent {

```

```

text = input<string>>('');
pause = input<boolean>(false);
stop = input<boolean>(false);
width = input<string>('100%');
height = input<string>('300px');

transcript = output<string>();

@ViewChild('videoA', { static: true })
videoA!: ElementRef<HTMLVideoElement>;

@ViewChild('videoB', { static: true })
videoB!: ElementRef<HTMLVideoElement>;

private active = signal<'A' | 'B'>('A');
private isRunning = false;

constructor(
  private emotion: EmotionService,
  private tts: TtsService,
  private sync: SyncEngine
) {

  // 🔥 preload all videos once
  [
    '/videos/explaining.mp4',
    '/videos/thumbs_up.mp4',
    '/videos/hand_raise.mp4'
  ].forEach(src => {

    const v = document.createElement('video');

    v.src = src;
    v.preload = 'auto';
  });

  effect(() => {
    this.sync.isPaused.set(this.pause());
    this.sync.isStopped.set(this.stop());
  });

  effect(() => {
    if (this.text()) {
      this.start(this.text());
    }
  });
}

```

```

resetVideos() {

  const videos = [
    this.videoA.nativeElement,
    this.videoB.nativeElement
  ];

  videos.forEach(v => {
    v.pause();
    v.removeAttribute('src');
    v.load();
    v.style.opacity = '0';
  });

  this.active.set('A');
}

preload(src: string) {
  const v = document.createElement('video');
  v.src = src;
  v.preload = 'auto';
}

switchVideo(src?: string, stop: boolean = false) {

  const current = this.active() === 'A'
    ? this.videoA.nativeElement
    : this.videoB.nativeElement;

  const next = this.active() === 'A'
    ? this.videoB.nativeElement
    : this.videoA.nativeElement;

  if (stop) {
    current.pause();
    current.style.opacity = '0';
    return;
  }

  if (!src) return;

  // avoid reload
  if (current.src && current.src.includes(src)) {
    return;
  }

  next.pause();

```



```

next.removeAttribute('src');
next.load();

next.src = src;
next.currentTime = 0;

next.onloadeddata = () => {

  next.play().then(() => {

    next.style.opacity = '1';
    current.style.opacity = '0';

    this.active.set(
      this.active() === 'A' ? 'B' : 'A'
    );

  }).catch(err => {
    console.error('Video play error:', err);
  });
};

next.onerror = (e) => {
  console.error('Video load failed:', src, e);
};
}

async start(text: string) {

  this.sync.isStopped.set(true);

  await new Promise(r => setTimeout(r, 50));

  this.sync.isStopped.set(false);

  this.resetVideos();

  if (this.isRunning) return;

  this.isRunning = true;

  try {

    const sentences =
      text.match(/[^.!?]+[.!?]+|^[^.!?]+$/g)
      || [text];

    const emotions =

```

```

        await this.emotion.analyze(text);

        await this.sync.play(
            sentences,
            emotions,
            this.tts,
            this.switchVideo.bind(this),
            this.preload.bind(this),
            (t: string) => this.transcript.emit(t)
        );

    } finally {
        this.isRunning = false;
    }
}
}

```

src/app/ai-video-agent/ai-video-agent.component.html

```

<div
  class="wrapper"
  [style.width]="width()"
  [style.height]="height()">

  <video #videoA muted playsinline></video>
  <video #videoB muted playsinline></video>

</div>

```

src/app/ai-video-agent/ai-video-agent.component.scss

```

.wrapper {
  position: relative;
  overflow: hidden;
}

```

```
video {  
  position: absolute;  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  
  opacity: 0;  
  transition: opacity 120ms linear;  
  will-change: opacity;  
}
```

Example Usage

```
<div style="width:400px;height:300px;">  
  
  <ai-video-agent  
    [[text]="response"  
    [[pause]="isPaused"  
    [[stop]="isStopped"  
    [[transcript]="onText($event)]>  
  </ai-video-agent>  
  
</div>
```

Install Azure Speech SDK

```
npm install microsoft-cognitiveservices-speech-sdk
```

Video Folder Structure

```
projects/c-bot/public/videos/
```

Files:

```
explaining.mp4  
thumbs_up.mp4  
hand_raise.mp4
```

Runtime URLs

```
/videos/explaining.mp4  
/videos/thumbs_up.mp4  
/videos/hand_raise.mp4
```